

UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ciencias

Escuela Profesional de Ciencia de la Computación

- **Curso: Fundamentos de Programación**
- **Docente:** Américo Chulluncuy Reynoso

2026-1

Sesión 9:

Memoria dinámica y arreglos dinámicos en C++

Contenido

1. ¿Por qué los arreglos estáticos no son suficientes?
2. El heap: memoria flexible en tiempo de ejecución
3. Reservando memoria con new
4. Liberando memoria correctamente con delete
5. Construcción de arreglos 1D
6. Modelando matrices dinámicas
7. RAII y Punteros Inteligentes
8. Herramientas para el manejo de memoria

1. ¿Por qué los arreglos estáticos no son suficientes?

```
const int CAPACIDAD = 10;
double a[CAPACIDAD];
```

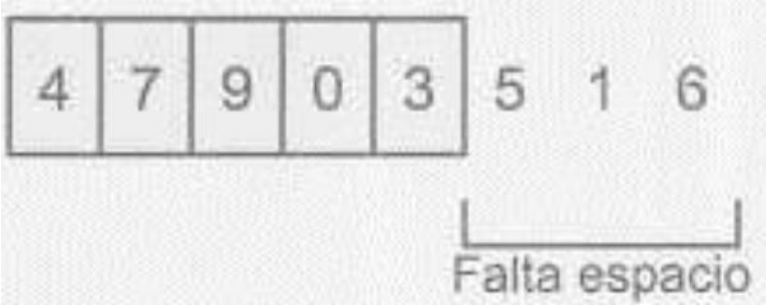
Tiene un **tamaño fijo**, definido en tiempo de compilación y no puede modificarse durante la ejecución del programa. Se almacena en el **stack**, lo que limita su tamaño (1-8MB aprox.)

Inconvenientes:

1) Desperdicia memoria, cuando el tamaño del arreglo excede el número de valores que se almacenarán en ella.

2) Riesgo de desbordamiento de pila, si se necesita almacenar más elementos de los que permite el tamaño fijo del del arreglo , se corre el riesgo de sobrescribir memoria.





Necesidad de la memoria dinámica

La memoria dinámica es útil en situaciones donde:

- El tamaño de los datos a almacenar no se conoce o varía en tiempo de ejecución
- Se desea optimizar el uso de memoria, asignando solo lo necesario
- Es necesario ampliar o reducir el tamaño del almacenamiento a medida que cambian las condiciones del programa.

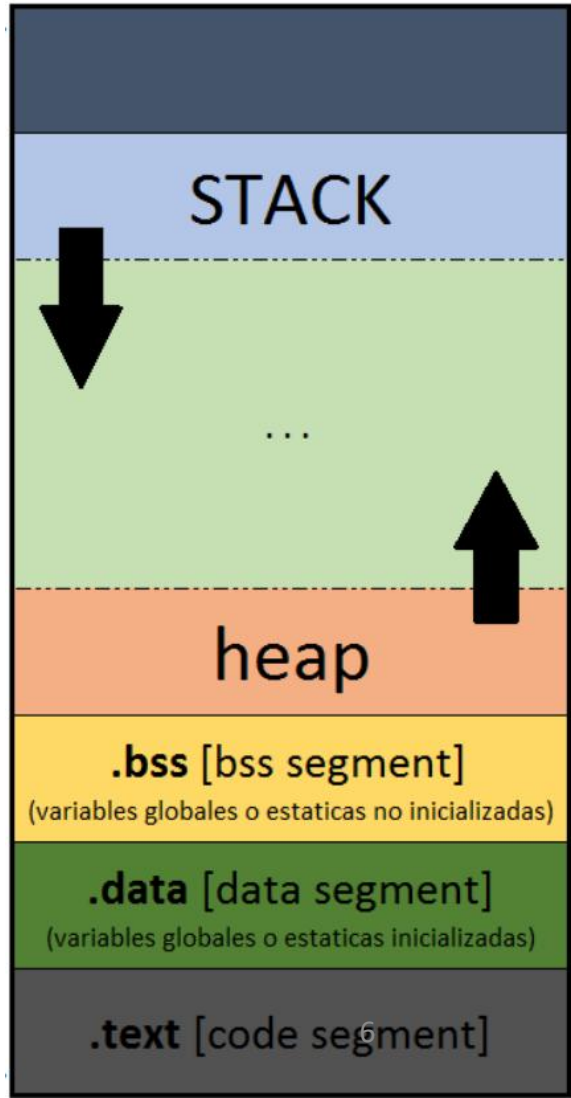
El uso de **memoria dinámica** en **C++ se gestiona de forma explícita**:

- La **asignación y liberación** se realiza mediante los operadores **new** y **delete** respectivamente.
- C++ moderno ofrece alternativas más seguras y recomendadas para la gestión de memoria.
  - Los Punteros inteligentes (Smart Pointers), automatizan la liberación de memoria.
  - Los contenedores dinámicos como std::string, std::vector, etc. manejan internamente memoria dinámica.

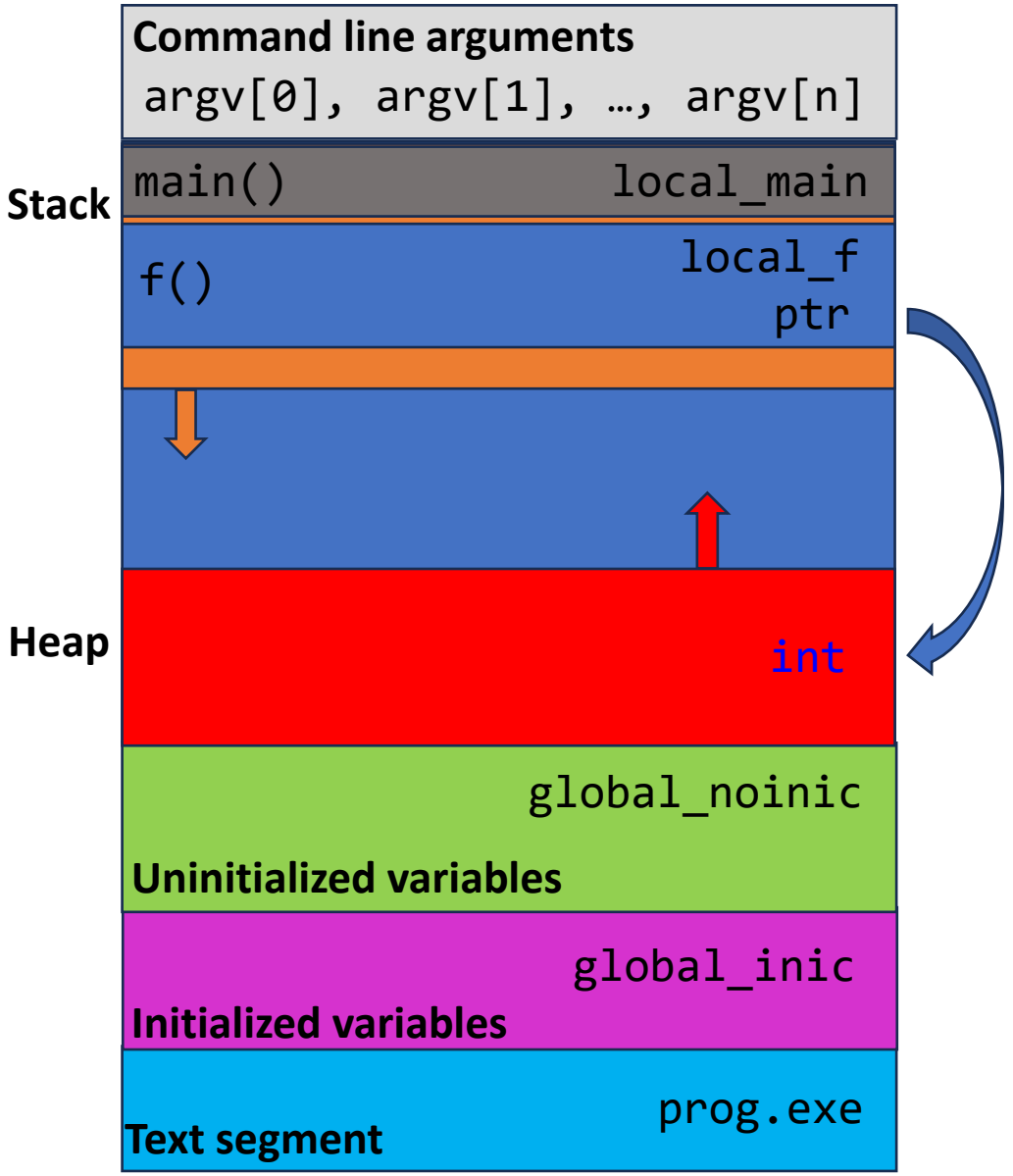
2. El heap: memoria flexible en tiempo de ejecución

Al ejecutar un programa, el SO le asigna un espacio de memoria (normalmente respaldado por RAM y memoria virtual) dividido en varias regiones principales:

1. **Área del programa** (text segment) contiene las instrucciones en lenguaje máquina del programa; suele ser de solo lectura.
2. **Segmento BSS** (Block started by symbol) almacena variables globales y estáticas no inicializadas.
3. **Área de datos** (data segment), almacena constantes y variables globales o estáticas.
4. **Pila** (Stack) maneja llamadas a funciones (registro de activación). Almacena parámetros de funciones, variables locales, direcciones de retorno. Su gestión es automática y muy eficiente.
5. **Montículo** (Heap) región destinada a la asignación dinámica de memoria. La memoria reservada permanece disponible hasta que el liberada explícitamente por el programana(delete)



Distribución de la memoria en un programa



prog.cpp → prog.exe

```
#include<iostream>
using namespace std;

int global_inic = 20; //segmento de datos
int global_noinic; //segmento bss (block started
                        by symbol)

void f(){
  int local_f; //stack (pila)
  int *ptr = new int; //heap
  *ptr = 20;
  delete ptr; //heap
  ptr = nullptr; // buena práctica

}

int main(int argc, char *argv[]){
    int local_main; //Stack

    f();
    return 0;
}
```

Operadores de gestión de memoria en C++

- C++ introdujo nuevas formas de gestionar la memoria dinámica (respecto a la gestión de memoria dinámica en C) con los operadores **new** y **delete**:

- **Ventajas** de new y delete:
  1. forman parte del lenguaje C++, no son funciones de una librería

```
new int;      versus  (int*)malloc(sizeof(int));
```

  2. No especificar tamaño, el compilador sabe el tamaño del tipo

```
new int[10]; versus  malloc(10 * sizeof(int));
```

  3. Inicialización en un paso, asignación + inicialización juntas

```
new int{42}; versus  malloc() + asignación aparte
```

- Importante: new y delete **NO son solo "syntactic sugar"** para malloc/free. new **llama al constructor** de los objetos (fundamental en C++ orientado a objetos). Esto se verá más adelante.

3. Reservando memoria dinámica con new

- **new** solicita memoria al SO en el heap (montículo) en función del tipo de dato. Devuelve la dirección (puntero) del bloque asignado. Los datos pueden ser:
  - ✓ **integrados** (primitivos): int, float, double, char, bool
  - ✓ **definidos por el usuario**: struct, class, enum.
  - ✓ Estructuras: arreglos, matrices, objetos STL (vector, string)

- **Sintaxis básica:**

```
int *ptr1 = new int;      // Asigna un int (sin inicializar)
int *ptr2 = new int{42}; // Asigna e inicializa a 42
int *arr = new int[n];    // Asigna un arreglo, n puede ser una variable
```

- Si no hay suficiente memoria, new lanza una excepción std::bad_alloc.

Manejo de errores y regla de oro de new

Siempre debes capturarla en entornos donde la memoria sea limitada. Una forma básica de hacerlo es int *p = new (std::nothrow) int; devuelve nullptr si falla.

```
if (p == nullptr) {
     // No hay memoria suficiente
}
```

- **Regla de oro**:
  - Cada new debe tener su correspondiente delete
  - Cada new[] debe tener su correspondiente delete[].

La memoria del heap no se libera automáticamente al salir de la función.

```
delete ptr1;     // Libera un solo elemento
delete ptr2;     // Libera un solo elemento
delete[] arr;    // Libera un arreglo
```

4. Liberando memoria correctamente con delete

- **delete ptr;** libera la memoria asignada con new y destruye el objeto
  delete[] ptr; libera un arreglo dinámico creado con new[]

- **Importante:** delete libera la memoria, pero el puntero sigue existiendo (sigue guardando la dirección antigua)

- **Reglas críticas:**
  1. Solo liberar memoria creada con new
  2. Coincidir new con delete y new[] con delete[]
  3. Nunca liberar 2 veces el mismo puntero (Double-free)

  Consecuencia: No respetar estas reglas =
  comportamiento indefinido(UB) + vulnerabilidad del sistema

Ejemplo

```
pt1 = new int;     // Asigna meoria para un solo entero
*pt1 = 130;
pt2 = new int[5]; // Arreglo de 5 enteros
delete pt1;        // Libera el bloque de pt1
delete[] pt2;      // Libera el arreglo de pt2

pt1 = nullptr;     // BUENA PRÁCTICA: Evitar punteros colgantes
                      (dangling pointers)
pt2 = nullptr;
```

Fugas de memoria

- Una fuga ocurre cuando pierdes la dirección de memoria dinámica antes de liberarla. Esa memoria queda secuestrada hasta que el programa termina. Si el programa sigue ejecutándose, la memoria se agota y el sistema colapsa.

```
char* s1 = new char [20];
char* s2 = new char[40];

strcpy(s1, "Memory leak");
s2 = s1;// FUGA!; 40 bytes no se pueden liberar

delete [] s2; // libera caja de 20 bytes
delete[] s1; // ERROR, double-free

s1 = nullptr; s2 = nullptr;
```

5. Construcción de arreglos 1D

- **Problema**: queremos almacenar notas de estudiantes, pero no sabemos cuántos serán.
  Solución: Asignación dinámica en 4 pasos:

1. Preguntar cuántos estudiantes hay
2. Reservar memoria justo para esa cantidad
3. Usar el arreglo como siempre (ejemplo: ingrese y muestre notas)
4. Liberar memoria

```
int *notas = new int[n];
 /* new reserva memoria en el HEAP.
    notas guarda la dirección inicial.
    Ahora el tamaño depende de n. */
```

Ejercicio 1

- Utilizando asignación dinámica y funciones. Escribir un programa que almacene las notas de n estudiantes del curso y calcule el promedio. A continuación, muestre el número de estudiantes cuya nota está por encima del promedio. Sugerencia de prototipo de las funciones

```
float* reservaMemoria(int );

void leerNotas(float*, int);

float promedio(float*, int);

void liberaMemoria(float*);
```

6. Modelando matrices dinámicas

1. Matriz contigua (máximo rendimiento) es un único bloque de memoria que almacena todos los elementos fila por fila (orden row-major)

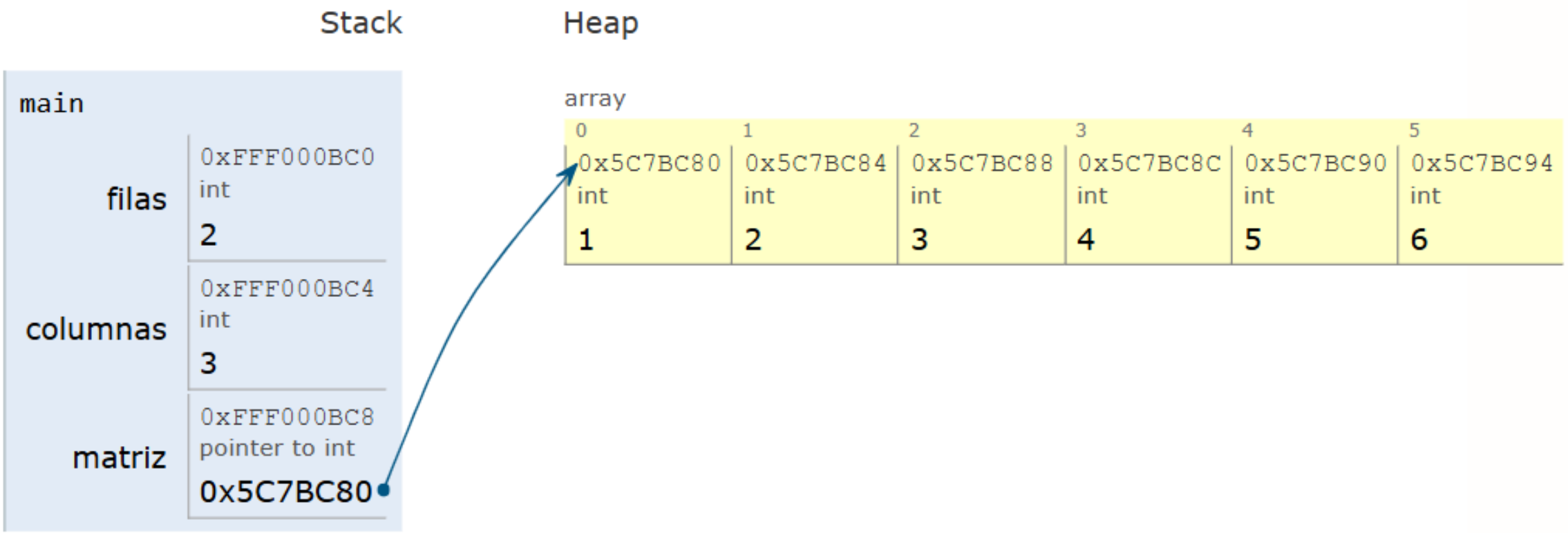
```
int filas = 2, columnas = 3;

// Reservar un solo bloque contiguo e inicializa
int *matriz = new int[filas * columnas]{1,2,3,4,5,6};

// Acceso con fórmula row-major (orden por filas)
matriz[i * columnas + j] = valor;  // Escritura
int dato = matriz[i * columnas + j]; // Lectura
```

Posición = (fila × número_de_columnas) + columna

Visualización: Matriz en memoria (orden por filas)



Usando un arreglo de punteros

2. **Matriz no contigua** (flexibilidad, menor rendimiento). Se reserva un arreglo de punteros (una fila por puntero) y cada puntero apunta a un bloque independiente (cada fila)

```
int filas = 2, columnas = 3;

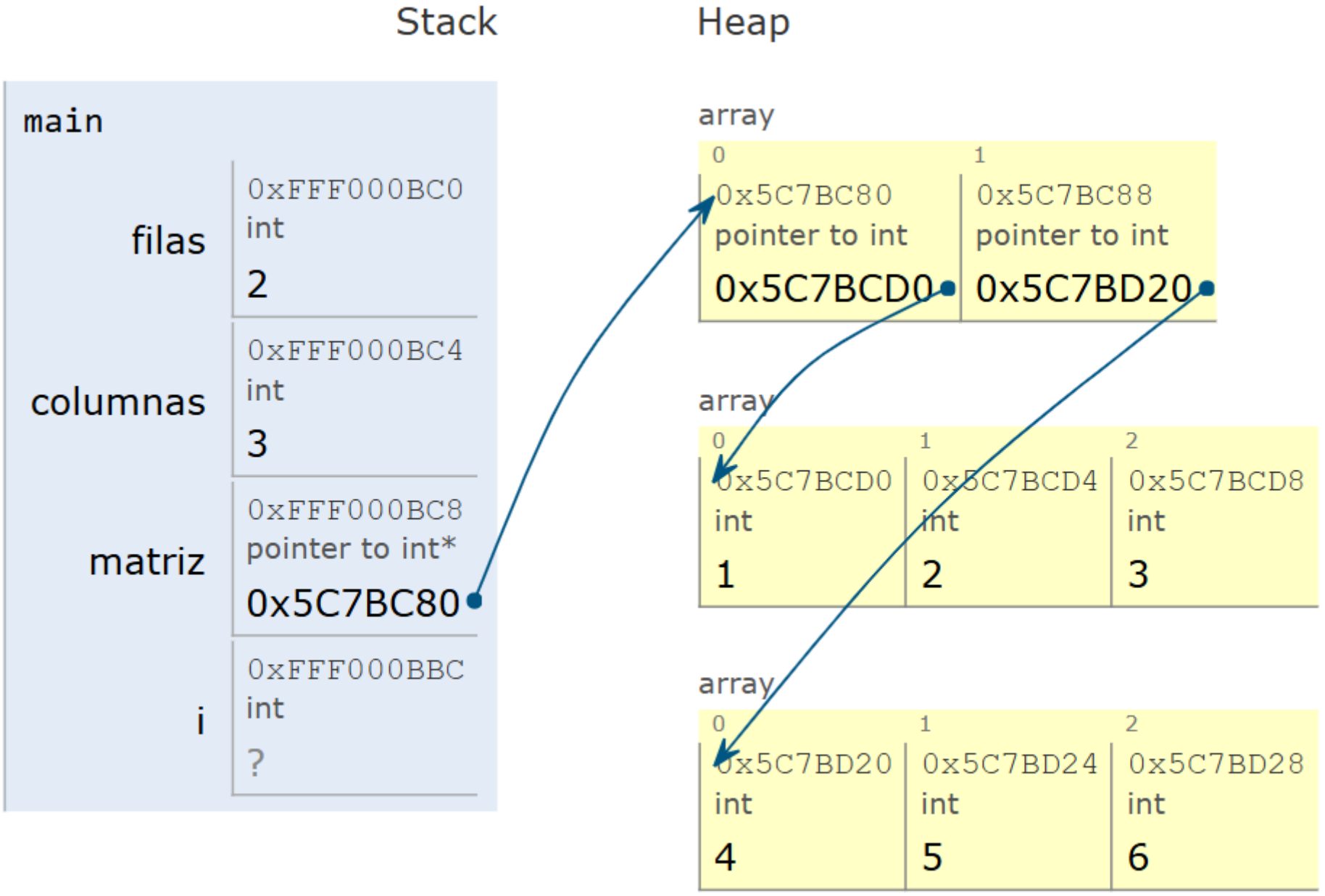
int **matriz = new int*[filas]; // Paso 1: Crear arreglo de punteros

for(int i = 0; i < filas; i++) { // Paso 2: Crear cada fila individualmente
    matriz[i] = new int[columnas];
}

matriz[i][j] = valor; // Acceso tradicional (más legible pero más lento)
int dato = matriz[i][j];

// Liberación (requiere bucle inverso)
for(int i = 0; i < filas; i++) {
    delete[] matriz[i];
}
delete[] matriz;
```

Visualización: Arreglo de punteros (filas no contiguas)



Ejercicio 2: Creando una matriz 3D

Una matriz 3D se puede visualizar como:

- **X** = profundidad (capas)
- **Y** = filas
- **Z** = columnas

Implementar una matriz dinámica 3D de tamaño [x][y][z] utilizando dos enfoques

- Forma 1: Utilizando puntero

- Forma 2: Utilizando puntero a puntero a puntero

7. Smart Pointers – Gestión automática de memoria

- **RAII** (Resource Acquisition Is Inicialization) los recursos se adquieren en el **constructor** y se liberan en el **destructor**. El compilador garantiza que el destructor se ejecute al salir del ámbito. Ejemplo: ver theory/RAII.cpp

- Los punteros inteligentes **son objetos RAII** que envuelven punteros nativos, cuando el Smart pointer sale de ámbito, **automáticamente libera la memoria**

- Evita delete manual (el destructor lo hace por ti). No más fugas (memory leak). No más punteros colgantes (dangling pointers).

- Ejemplo: los 3 tipos de Smart pointers

8. Herramientas para detectar errores de memoria

- ¿Por qué necesitas herramientas?
  - Las fugas de memoria son difíciles de detectar
  - El compilador NO advierte sobre double-free
  - Los errores de memoria son la principal causa de bugs en C++

- **Herramienta 1: AddessSanitizer (ASan),**
  - **Integrado GCC y Clang**
  - **Activación:** -fsanitize=address
  - **Detecta en tiempo de ejcución**: Buffer overflow, use-after-free, double-free, memory leaks
  - Rápido

- **Herramienta 2: Valgrind (Linux)** Clásica y potente ideal para análisis profundos**.**
  - Herramienta clásica de Linux, WLS
  - **Activación**: valgrind ./programa (sin recompilar)
  - **Detecta:** Memory leaks, memoria no inicializada
  - Lenta

REGLA DE ORO: Si compila sin Asan, pero falla con Asan, tienes UB